

Homework5 2023202300 张昕跃

Q1(1)

Assume we have following address binding table and value of registers :

Address	Value	Register	Value
0x100	0x10	%eax	0x10
0x110	0x11	%ebx	0x100
0x120	0x12		
...	...	...	...
0x190	0x19		
0x200	0x20		

Answer1(1)

Please fill in the table below

Operand	Value
%ebx	0x100
\$0x150	0x150
0x170	0x170
(%ebx)	0x10
(%ebx,%eax)	0x11
0x30(%ebx)	0x13
80(%ebx,%eax,2)	0x20

- 1. 寄存器取值（例如%eax、%ebx）
- 2. 立即数取值（\$+数字）

3. 地址取值（0x170地址中的值应该是0x17）
4. 寄存器+内存，（）中是内存地址，同时被寄存器存储
5. %ebx + %eax的值，即0x110中的值
6. 立即数+内存+寄存器，0x30是偏移地址，因此是0x130存储的值
7. $80 + \%ebx + 2 * \%eax$ 存储的值，也就是0x200中的值

Q1(2)

Suppose registers and bound values will be reset as above after each instruction. Please fill in the table below: (Write all if there are more than one destinations and None if there is no destination)

Answer1(2)

Instruction	Destination	Value
addl %eax,%ebx	%ebx	0x110
subl %eax,(%ebx)	0x100	0x0
leal 0x50(%eax), %edx	%edx	0x60
movzbl %al, %ebx	%ebx	0x00000010
movsbl %bh, %ecx	%ecx	0x00000001

1. 把%eax中的值加到%ebx中去
2. 把%eba寄存器中的值取出来作为内存地址，将内存地址对应的值减掉%eax寄存器中的值
3. leal是将地址加载到寄存器中，这里就是将%ebx中的值+0x50传到%edx中
4. movzbl负责拷贝一个字节（%al寄存器，%eax的低八位），并用0填充其目的操作数中的其余各位，完成“零拓展”，拓展为四个字节（%ebx是32位寄存器）
5. movsbl和movzbl类似，不过是完成“符号拓展”，操作%ebx的高八位复制到%ecx中去

Q1(3)

Assume the initial value of the flags is 0. Fill the table below

Answer1(3)

Instruction	OF	SF	ZF	CF
leal(%eax),%ebx	0	0	0	0
subl %ebx, %eax	0	1	0	1
xorl %eax, %eax	0	0	1	0
test %eax, %ebx	0	0	1	0

- OF(Overflow Flag)
- SF(Sign Flag)
- ZF(Zero Flag)
- CF(Carry Flag)

1. 将(%eax)中的有效地址(这里就是寄存器中的值0x10)加载到%ebx，不会改变符号位，产生进位等等
2. 将%eax中的值减去%ebx中的值，也就是0x10-0x100，会影响符号位SF，借位也会影响CF
3. 将%eax寄存器中的值与其本身异或，结果为0，影响ZF
4. test是按位与操作，也就是0x10&0x100，结果也为0，影响ZF

Q2

- Translate the following assembly into C codes.
- You can name local variables represented by -12(%ebp), -8(%ebp)...or a,b,c... freely as you like.
- The beginning of C codes is given.

```

push    %ebp
movl    %esp,%ebp
subl    $0x10, %esp
movl    $0x3,-0xc(%ebp)
movl    $0x2,-0x8(%ebp)
movl    $0x1,-0x4(%ebp)
jmp     .L1
.L2:
movl    -0x4(%ebp),%eax
movl    %eax,-0x10(%ebp)
movl    -0x8(%ebp),%eax
movl    %eax,-0x4(%ebp)
movl    -0x10(%ebp),%eax
addl    %eax,-0x8(%ebp)
addl    $0x1,-0xc(%ebp)
.L1:
cmpl    $0x5,-0xc(%ebp)
jle     .L2
movl    -0x8(%ebp), %eax
leave
ret

```

```

int -0xc(%ebp) = 3;           int i = 3;
int -0x8(%ebp) = 2;          or  int b = 2;
.....

```

Answer2

```

// 寄存器写法
int -0xc(%ebp) = 3; // movl $0x3,-0xc(%ebp) / int i = 3
int -0x8(%ebp) = 2; // movl $0x2,-0x8(%ebp) / int b = 2
int -0x4(%ebp) = 1; // movl $0x1,-0x4(%ebp)
// 随后跳转到 .L1
// cmpl $0x5,-0xc(%ebp) 检查 -0xc(%ebp) 是否为(<=) 5
// 如果满足, 跳转到 .L2
while( -0xc(%ebp) <= 5)
{
    int %eax = -0x4(%ebp);           // movl -0x4(%ebp), %eax
    int -0x10(%ebp) = -0x4(%ebp);   // movl %eax, -0x10(%ebp)
}

```

```
%eax = -0x8(%ebp);           // movl -0x8(%ebp), %eax
-0x4(%ebp) = -0x8(%ebp);      // movl %eax, -0x4(%ebp)
%eax = -0x10(%ebp);           // movl -0x10(%ebp), %eax
-0x8(%ebp) += -0x10(%ebp);     // addl %eax, -0x8(%ebp)
-0xc(%ebp)++;                 // addl $0x1, -0xc(%ebp)
}

%eax = -0x8(%ebp);           // movl -0x8(%ebp), %eax
return %eax;
```

```
// 变量写法
int i = 3;
int b = 2;
int c = 1;

while(i<=5){
    int temp = c;
    c = b;
    b += temp;
    i++;
}

return b;
```